

Job Sequencing and Tool Switching

Structural results and a branch-and-Benders-cut study

Shankar Narayanan

Advisors: Prof. Rafael Colares Prof. Annegret Wagler
Academic supervisor: Dr. Renaud Chicoisne

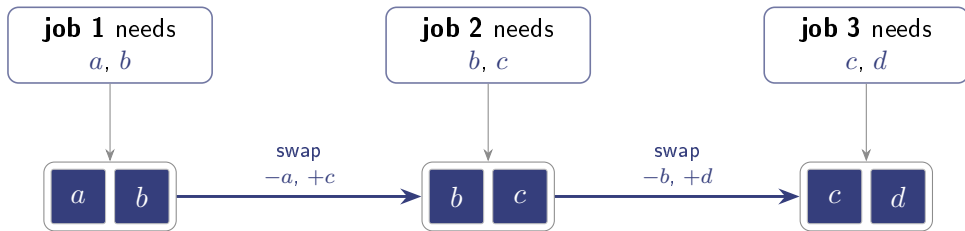
Laboratory of Informatics, Modelling and Optimisation of Systems (LIMOS)
ISIMA, Clermont Auvergne INP

Internship defence 2 September 2026



- 1 The problem
- 2 State of the art, and what was open
- 3 Structural results
- 4 A branch-and-Benders-cut algorithm
- 5 Strengthening the solver
- 6 Computational results
- 7 Why the bound does not move
- 8 Limitations
- 9 Conclusions and further work

A machine, a magazine, and a set of jobs

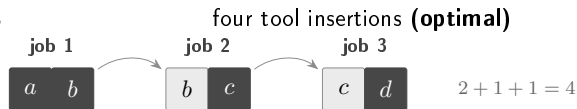


the magazine (grey box) holds only $b = 2$ tools at a time

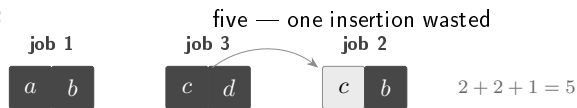
- A machine can hold only b tools at a time. Each job needs its own subset.
- Running the jobs in some order means **stopping to change tools**, and each stop costs production time.
- **The problem.** Choose the order of the jobs, and the tools loaded at each step, so that the total number of tool insertions is smallest.

The order decides how many tools come back

order 1, 2, 3



order 1, 3, 2



inserted here (counted)



already loaded (free)

Same three jobs, same magazine, same loading rule. Only the order changed. Running job 2 in the middle keeps its tool loaded; running it last forces that tool to be **removed and inserted again**.

That second insertion is called a reinsertion. Everything in this talk is about where reinsertions come from and why they are hard to rule out.

One decision is easy, the other is not

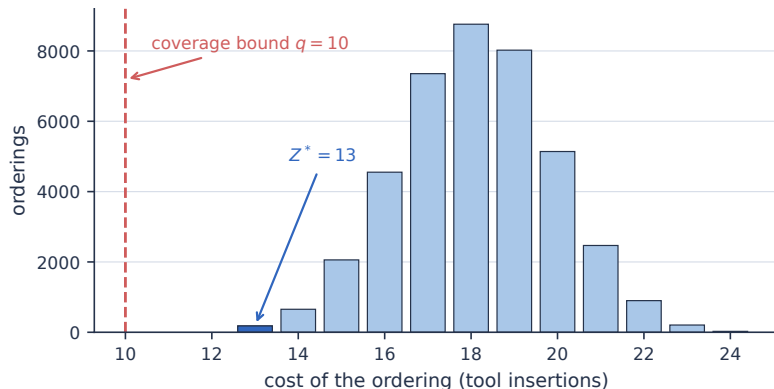
Loading is easy. Once the order is fixed, the cheapest loading is known in closed form: when a tool must enter a full magazine, remove the one whose next use lies furthest ahead. This is the *Keep Tool Needed Soonest* rule, it is optimal, and it runs in $O(n|T|)$ time (**Tang & Denardo, 1988**).

Ordering is hard. The problem is NP-hard for every fixed magazine capacity $b \geq 2$ (**Crama et al., 1994**). A good order puts jobs with similar requirements next to one another — but “similar” means only *overlapping sets*, with no natural distance, and the capacity couples jobs that are far apart in the order.

Why this shape matters

An **easy inner problem inside a hard outer one** is exactly what decomposition methods are built for. That is the reason this problem has attracted exact methods for four decades, and it is the starting point of the second half of this talk.

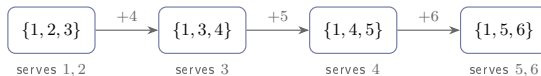
How hard is the ordering, concretely?



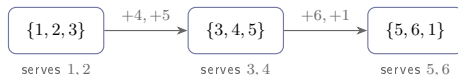
Every one of the 40,320 orderings of one eight-job benchmark instance, by cost. The optimum is 13 and only 180 orderings attain it. The elementary counting bound q is 10 — three below the optimum, and no ordering ever reaches it.

Where the exact methods stop

four configurations: cost 3



three configurations: cost 4



Compact integer models have been proposed for four decades (**Laporte et al., 2004; Catanzaro et al., 2015**). The strongest of those surveyed here relaxes to $q = |U| - b$: the number of tools used minus the magazine capacity (**da Silva et al., 2021**).

That is an **elementary counting argument** — every tool must enter at least once, and at most b can be there before the first job. A solver relying on it cannot prove optimality where that count is wrong.

Industry does not solve this problem directly. It groups the jobs into as few magazine-fitting batches as possible, then orders the batches. It is fast, and where it has been measured against exact optima in industrial printing it does well (**Burger et al., 2015**).

The construction is known to be a b -approximation (**Crama & van de Klundert, 1999**).

What had *not* been done:

- sharpen that ratio to something that depends on the instance in front of you;
- say on which instances the construction is **exact**;
- distinguish the two different values the method can be read as returning.

The third turns out to change the first two.

Two questions, and why only these two

Question 1 — structural

What does insisting on the *fewest* groups cost, once the final loading step is allowed to re-optimize?

Answering it needs no solver: it is a question about the construction itself.

Question 2 — algorithmic

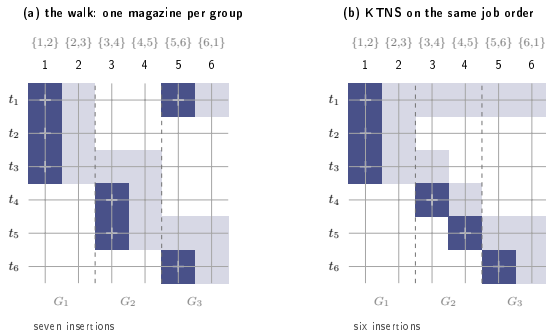
Every compact model surveyed stops at the counting bound q . Can a method that *never forms* that relaxation get past it?

Two classical routes qualify, for opposite reasons.

Why these two and no others

The first is the gap the literature left open about the method industry actually uses. The second follows from the problem's shape — an easy loading problem inside a hard ordering problem — which is precisely the precondition for **decomposition**. Everything else in the exact literature already sits below q .

The construction admits two different readings



(a) One magazine per group.

The reading the literature's worst cases are built on.

On the family most often used as an example, (a) pays one extra insertion and (b) pays **none**. Separating the two is a prerequisite for saying anything correct here.

(b) Re-optimize at the end.

What the method as specified actually returns.

The loss has a single cause. Allowing *all* feasible groupings makes the construction exact — the optimum equals the cheapest walk over every grouping. So the loss is not a defect of the method; it is the price of insisting on the fewest groups.

Guarantees that depend on the instance. A ratio bound $\min(b, K^* - 1, q)$ holding uniformly over every admissible grouping and tie-breaking order, families where the construction is provably exact for every capacity, and the extremal behaviour settled at three groups.

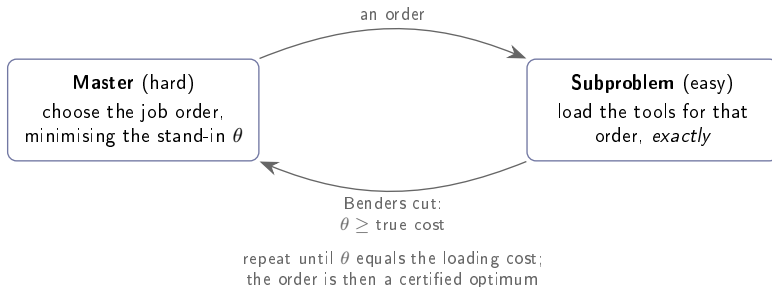
The headline. There is a **connected** family with

$$|U| = 8g + 1, \quad K^* = 3g, \quad Z^* = 8g - 5, \quad H = 9g - 5,$$

so the additive loss is g and grows without bound. The *ratio* tends to $9/8$, so a constant factor is not disproved.

This answers the question left open by **(Crama & van de Klundert, 1999)** for the method as it is actually specified.

The decomposition



The master chooses a job order using an optimistic stand-in θ for the loading cost. The subproblem returns the **exact** loading cost of that order. If θ undershot, one cut lifts it — and that cut is valid for *every* order, not only the current one.

Because the subproblem is polynomial, the master's bound is assembled from true subproblem values, never from a relaxation of the loading — the logic-based (**Hooker & Ottosson, 2003**) and combinatorial (**Codato & Fischetti, 2006**) Benders lineage.

The master problem

Introduce a depot d with no requirement and arc variables $x_{ij} \in \{0, 1\}$, so the x describe a Hamiltonian path through the jobs.

$$\begin{aligned} \min \theta \quad \text{s.t.} \quad & \sum_{i \neq j} x_{ij} = 1 \quad (\forall j), & \sum_{j \neq i} x_{ij} = 1 \quad (\forall i), \\ & \sum_{i,j \in S} x_{ij} \leq |S| - 1 \quad (S \subseteq J). \end{aligned}$$

Two bounds the master starts from. The *coverage row* $\theta \geq |U|$, and a *pairwise row*

$$\theta \geq \sum_{i,j \in J} w_{ij} x_{ij} + \sum_{j \in J} |T_j| x_{dj}, \quad w_{ij} = \max(0, |T_i \cup T_j| - b),$$

counting what two adjacent jobs must differ by. Both are visible to the linear relaxation; keep that in mind for later.

The subproblem: loading for a fixed order

Fix a candidate order $\bar{x}$. Let $y_{j,t} = 1$ if tool t sits in the magazine while job j runs, and let $z_{j,t}$ charge an insertion of t at j .

$$\begin{aligned} \min \quad & \sum_{j,t} z_{j,t} \\ & y_{j,t} \geq 1 & t \in T_j & (\nu_{jt}) \\ & \sum_t y_{j,t} \leq b & \forall j & (\mu_j) \\ & z_{j,t} \geq y_{j,t} - y_{i,t} - (1 - \bar{x}_{ij}) & \forall i \neq j, t & (\lambda_{ijt}) \\ & z_{j,t} \geq y_{j,t} - (1 - \bar{x}_{dj}) & \forall j, t & (\lambda_{jt}^d) \end{aligned}$$

The third row charges an insertion when a tool is present at j but absent at its predecessor i . The fourth does the same for the first job, charging its whole requirement, **because the machine starts empty**.

Integrality

For fixed $\bar{x}$ the master variables appear only on the right-hand side, so what remains is a linear program in y and z alone. Its constraint matrix has the **consecutive-ones property** along the path $\bar{x}$ selects, and is therefore totally unimodular. The relaxation is integral, and its optimum is exactly $\text{KTNS}(\bar{x})$ (**Tang & Denardo, 1988**).

Why this matters for the whole method. A Benders scheme is only as good as its subproblem. Here the subproblem is not approximated and not relaxed — it returns the true cost of the proposed sequence. So when the solver holds a good incumbent, that incumbent has been evaluated *exactly*.

What remains in question is whether the master accumulates a strong enough **lower bound** to prove no better order exists. That is the rest of this talk.

The optimality cut, in full

Only $\bar{x}$ appears on the right-hand side, so in the dual it appears only in the objective. Taking an optimal dual solution $(\bar{\lambda}, \bar{\lambda}^d, \bar{\mu}, \bar{\nu})$:

$$\theta \geq \sum_{i \neq j} \sum_t \bar{\lambda}_{ijt} (x_{ij} - 1) + \sum_j \sum_t \bar{\lambda}_{jt}^d (x_{dj} - 1) - b \sum_j \bar{\mu}_j + \sum_j \sum_{t \in T_j} \bar{\nu}_{jt}$$

Valid for every order.

The dual feasible region does not depend on $\bar{x}$, so the same cut holds at every point of the master.

Tight at this one.

At $\bar{x}$ it reproduces the exact loading cost. It is affine in x , so separating it at fractional points needs no extra argument.

The depot term was initially omitted

Dropping $\sum_{j,t} \bar{\lambda}_{jt}^d (x_{dj} - 1)$ makes the cut **too tight**: it silently assumes the first job's tools are free, and it cuts off genuine optima.

How it surfaced

Random instances were solved twice — once by brute force, once by the solver. The two disagreed on **193** of them, always with the solver's answer too high.

This is why the validation described later runs against an *independent* implementation, not against the solver itself.

What was done

The term was restored, the derivation was rewritten from the implementation rather than from the draft, and the **entire campaign was re-run**.

Four strengthenings, each a standard remedy

All four are classical elements of the Benders toolbox surveyed by (**Rahmaniani et al., 2017**). Each is an independent switch, checked against brute force *before* any campaign run, so a strengthening can never silently return a wrong optimum.

- **Fractional Benders cuts** — separate the same cut at fractional points, not only at integer orders.
- **A strong incumbent at the root** — a hybrid genetic search, used both as a warm start and to seed a cut.
- **A conflict-graph bound** — a combinatorial row the linear relaxation cannot see.
- **Pareto-optimal cut lifting** — when the dual has many optima, choose the deepest cut.

Two attack the **dual bound**. One attacks the **incumbent**. One chooses *among* cuts. Watch which kind helps.

The intention. An integer-only scheme learns about the loading cost only once an order is fully committed. A fractional cut extracts the same dual information from a *partial* order and removes it from the relaxation at once. This is the standard remedy for tailing-off, and it is legitimate here because the cut is affine in x .

It did what it should, locally

67.5 million cuts generated.

Median node count fell from **77,439** to **3,701** — a factor of twenty.

And the bound never moved

Across 1,404 paired runs the dual bound at termination was

higher	0
equal	1,226
lower	178

and **69** instances were lost.

The remedy is being applied correctly, to the wrong obstacle.

The intention. A branch-and-bound search prunes in proportion to the quality of its incumbent. A compact form of the hybrid genetic search of (Mecler et al., 2021) is run once, at the root.

Constructive seeds — nearest-neighbour orders on the pairwise weights w_{ij} , and a largest-tool-first order — each improved by a variable-neighbourhood local search using adjacent swaps and short segment relocations, every candidate scored by the exact loading oracle, evolved under order crossover with a diversity guard.

The order is then used twice. As a warm start it gives the solver an incumbent from node zero. And it seeds a cut: the loading dual is solved once for the heuristic order and its cut added before the first relaxation — the initial-cut-pool acceleration of (Rahmaniani et al., 2017).

This is the only strengthening aimed at the **primal** side, and the only one that helps: **+31** instances.

The intention. The master's root bound is pairwise plus coverage, and both are visible to the linear relaxation. The combinatorial structure they miss lives in the *conflict graph* G : vertices are jobs, edges join pairs that cannot share a magazine.

A clique of G is a set of pairwise incompatible jobs, which must occupy that many distinct configurations, so $K^* \geq \omega(G)$; more generally $K^* \geq \chi(G)$, and where G is non-bipartite $\chi(G) \geq 3$ even when $\omega(G) = 2$. The resulting row

$$\theta \geq (\chi_{\text{lb}} - 1) + \min(b, |U|)$$

is added whenever it strictly exceeds the coverage row. This is the simplest of the conflict-graph inequalities studied by (Atamtürk et al., 2000).

Its reach is narrow by construction. On most instances the coverage row already dominates it. It can bite only where the *grouping* structure, rather than the tool count, forces the switches. Measured effect: -1 instance.

The intention. When the loading dual has several optimal solutions — which a degenerate transportation-type problem routinely does — the cuts they yield are not equally strong. Choosing the deepest is the Pareto-optimal cut of **(Magnanti & Wong, 1981)**: among alternative dual optima, select the one maximising the cut's value at a fixed interior core point of the master.

(Papadakos, 2008) makes this practical by observing that the core point need not be tied to the current relaxation, and may simply be updated as a convex combination of the points visited. That form is used here: a core point is kept in the relative interior, initialised from the heuristic order blended with the barycentre, and drawn halfway towards the current relaxation point after each separation.

The core-point cut is added *in addition to*, never instead of, the ordinary cut, which keeps the scheme valid by construction.

Measured effect: —**37** instances — a deeper cut inside the wrong family.

What was compared

Nine Benders regimes and three reimplemented compact baselines: the tool-state model (**Laporte et al., 2004**), Formulation 4 (**Catanzaro et al., 2015**), and the multicommodity model (**da Silva et al., 2021**).

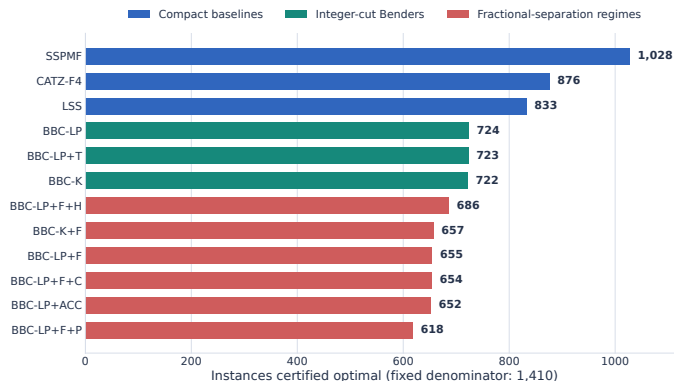
Under one protocol

1,410 canonical instances $\times$ 12 configurations = 16,920 planned outcomes, all present. One hour per run, one homogeneous node type, and a *fixed* denominator: every planned pair counts once, and any non-optimal outcome counts as not closed.

Two caveats stated before the numbers

Benders ran on one solver thread while the compact models ran on four; and F4 stands in for the stronger Formulation 5 that its authors recommend. Both run *against* this report's own conclusion, not for it.

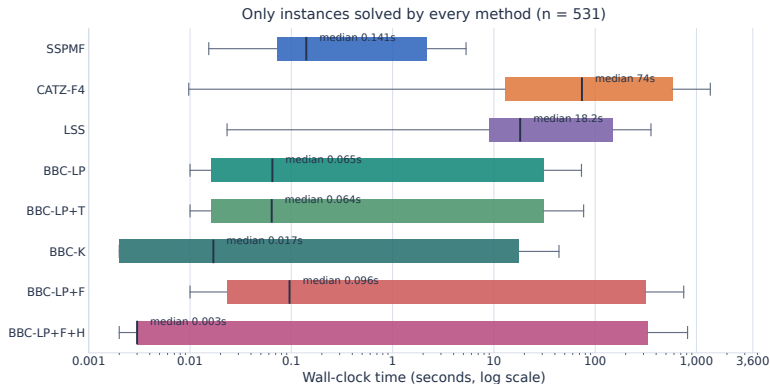
How far each method gets



Against the same 1,410-instance denominator the compact baselines certify more than every Benders regime. **This does not go the way I hoped**, and with the two caveats above the margin is if anything understated.

The rest of this section is about *why*, and the reason is not the search.

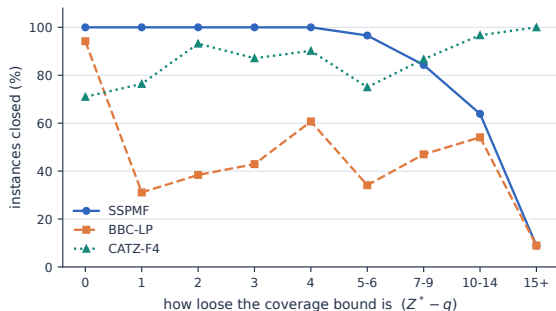
Solving times



Restricted to the 531 instances every method solves, the picture inverts: the Benders medians sit orders of magnitude below the compact ones. The axis is logarithmic.

So this is not a slow method. It closes an instance almost at once, or it does not close it within the hour — the signature of a bound that is either exactly right or of no use.

What separates the two cases



Group the 1,107 instances of known optimum by how far the optimum sits above the counting bound. Where the bound is exact the Benders solver certifies 94%; one unit above, 31%; then no trend at all.

But this is a fact about the method, not the instances. F4 moves the *other* way and closes all 45 of the loosest. And the multicommodity model's relaxation is also q , yet it loses only 13 points. What loses 54 is a master carrying the coverage row and, at the root, almost nothing else.

The solver usually already has the answer

Among time-limited runs whose optimum is known from elsewhere, the Benders incumbent **already equals that optimum** in 223 of 383 cases. The median remaining gap is nonetheless 23.1%.

Put beside the ablation, the pattern is consistent:

- the one strengthening aimed at the **incumbent** gains instances;
- every strengthening aimed at the **dual bound** is neutral or harmful;
- 67.5 million fractional cuts raise the bound at termination on *no* instance.

A solver whose difficulty was *finding* good orders would not behave like this.

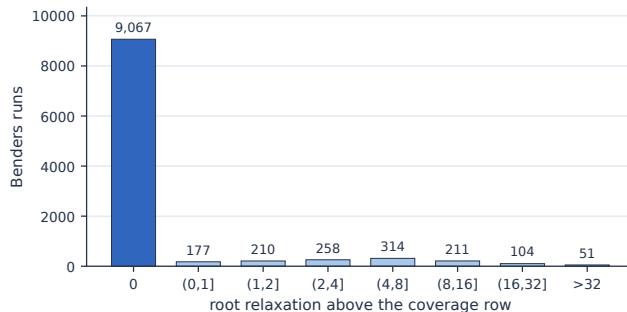
Write β_t for the number of maximal stretches during which tool t sits in the magazine. Every insertion of t begins one, so

$$Z^* = \sum_{t \in U} \beta_t, \quad \text{and} \quad \beta_t \geq 1 \text{ for every used tool}$$

recovers exactly the counting bound q . **Everything above q is a reinsertion:** a tool evicted and brought back.

A reinsertion happens when the jobs needing a tool are pushed apart by other jobs whose requirements cannot sit alongside it. That is a property of the **global interleaving** of the schedule. It is not a property of which pairs of jobs happen to be adjacent.

And every inequality the solver carries is local



The master carries one row indexed by *arcs*, triplet rows that are the weak linearisation of a product, and a constant coverage row. Every optimality cut it generates is also arc-supported, with coefficient $\sum_t \lambda_{ijt}$ on x_{ij} .

So the master learns only **adjacency-indexed** facts about a cost that is not adjacency-indexed. Measured: on 87.2% of runs the root relaxation equals the coverage row exactly — the pairwise and triplet rows contribute nothing at all.

A family that is not local, and what it was worth

For a stretch of positions $W = [p, r]$, every tool needed inside W is either already loaded at $p - 1$ or inserted within W :

$$\sum_t \sum_{k \in W} w_{t,k} \geq |\{t \text{ needed in } W\}| - \sum_t a_{t,p-1}.$$

Valid, and it counts across a *span* rather than an arc. Its ceiling over integral schedules is several times that of the family the solvers carry today.

And it delivered nothing

Implemented and measured inside the master, it left the root relaxation unchanged on all 37 instances for which both values were obtained — because its fractional row turns out to be **implied** by rows the formulation already has.

Writing a global inequality down is necessary. It is not sufficient.

Ordered by how much each threatens the conclusions just drawn.

- **The census of heuristic gaps is small.** The one fixed random census is 1,260 instances at three to five jobs. The connected family settles *existence* and unboundedness; it says nothing about how often such gaps occur at industrial sizes.
- **A ceiling over integral schedules is not a bound.** The window family's headroom was measured by enumerating every sequence of 62 loose cases at eight jobs. That is a screen, not an estimate — and the implemented row raised no root.
- **The benchmark directory was not the published benchmark.** It held 171 files against the 160 defined; ten duplicate canonical cases byte for byte and one is a smoke test. All eleven are filtered explicitly, and the count is checked before any figure is computed.

- **One hour, one machine, one collection — and F4, not F5.** Every solve count comes from a single limit on one node type, and the compact baseline is weaker than the best available. Both cut *against* the conclusion drawn here, and neither touches the relaxation values.
- **The branch-and-price prototypes are Python; the baselines are C.** A pricing node costs orders of magnitude more than a node inside a commercial solver. Nothing above nine jobs was closed, and that carries no information about the formulations — which is why no conclusion here rests on a branch-and-price solve count, only on a root value recorded before any search.
- **The learned cut-selection prototype was never run on this problem.** It was trained on knapsack cover cuts. It shows that the agent learns; it says nothing about the SSP, and it is reported as an appendix for that reason.

One equivalence organises the whole report: a schedule is a **walk through magazine configurations**, and its cost is the length of that walk.

Structural

The two-phase construction has two readings; separating them is a prerequisite for saying anything correct about it.

The optimum is the cheapest walk over *all* groupings — the gap is exactly the price of minimum cardinality.

A ratio bound $\min(b, K^* - 1, q)$ holds for *every* instance, and zero-gap classes exist at every capacity.

For the method as specified: four jobs are necessary and sufficient for a positive gap, and a *connected* family proves it unbounded.

Algorithmic

A branch-and-Benders-cut solver whose subproblem is exact, polynomial, and never relaxed.

Position-indexed formulations with a fixed polynomial row set, one of which *provably* exceeds the coverage bound.

Across 16,920 outcomes, four independent measurements agree: the obstacle is the **bound**, not the search.

Bound-limited, not implementation-limited — a negative result about a family of methods, not one solver.

The direction the campaign points to hardest

A bound that reaches into the loose half. The arc-supported information is exhausted; the honest next step is a global family whose *fractional* encoding is not implied by rows the formulation already has — which is exactly where the window family failed.

- **Two concrete strengthenings, both untested.** Chain PTF per configuration rather than per tool — that is what confines its advantage to two runs. And disaggregate the master's single θ into one variable per position, the standard remedy the ablation never tried.
- **The exact worst case of the heuristic as specified.** Settled at three groups for the walk value; open for the method itself. Two concrete questions: is the gap ever larger than one on a connected instance, and does $\text{gap} \leq K^* - 2$ hold? Directed searches at $b \in \{3, 4, 5\}$ found no counterexample to either.

- **Non-uniform tool costs, for free.** The subproblem is the loading linear program, not the KTNS rule, and that program stays exact when tools have different setup times while the rule does not (**Crama et al., 1994**). The solver therefore extends unchanged to a regime the whole SSP literature assumes away, and testing it there would broaden the method's scope.
- **Beyond the SSP.** Nothing in the position–transition construction is specific to tool switching except the transition metric; it applies to any generalised travelling-salesman problem with overlapping clusters (**Pop et al., 2024**). Whether the relaxation strength observed here survives that generality is open.
- **Where a learned component could pay.** Not cut selection — a selector cannot select a cut that does not exist. Column selection for the position-indexed pricers, whose coupled-pair oracle is the measured bottleneck, is the target with a chance of helping.

The cheapest loading for a fixed order is known in closed form, and the rule that computes it is optimal in $O(n|T|)$ time.

Tang, C. S. & Denardo, E. V. (1988). Models arising from a flexible manufacturing machine, Part I. *Operations Research* 36(5), 767–777.

The ordering is NP-hard for every fixed capacity $b \geq 2$, and the loading linear program stays exact when tool costs differ while the rule does not — which is why the solver built here extends to that case.

Crama, Y., Kolen, A. W. J., Oerlemans, A. G. & Spieksma, F. C. R. (1994). Minimizing the number of tool switches on a flexible machine. *IJFMS* 6, 33–54.

The two-phase construction industry uses is a b -approximation — the worst-case result this report sharpens into an instance-sensitive theory.

Crama, Y. & van de Klundert, J. (1999). Worst-case performance of approximation algorithms for tool management. *Naval Research Logistics* 46, 763–783.

The one industrial study measuring that construction against exact optima found it performing well in multi-colour printing — the study the censuses here agree with.

Burger, A. P., Jacobs, C. G., van Vuuren, J. H. & Visagie, S. E. (2015). Scheduling multi-colour print jobs with sequence-dependent setup times. *Journal of Scheduling* 18, 131–145.

The tool-state model, the first exact treatment and one of the three baselines reimplemented here. Laporte, G., Salazar-González, J. J. & Semet, F. (2004). Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions* 36, 37–45.

The strongest compact family surveyed, and the source of the counting bound q that governs everything in the second half of this talk. Formulation 4 stands in here for the Formulation 5 its authors recommend, so the baseline is weaker than the best available.

Catanzaro, D., Gouveia, L. & Labbé, M. (2015). Improved integer linear programming formulations for the job sequencing and tool switching problem. *EJOR* 244, 766–777.

The multicommodity model, whose relaxation is also q — yet which loses only thirteen points across the loose instances. That is the measurement showing the coverage bound alone does not explain the Benders solver's failure.

da Silva, T. T., Chaves, A. A. & Yanasse, H. H. (2021). A new multicommodity flow model for the job sequencing and tool switching problem. *International Journal of Production Research* 59, 3617–3632.

Key references — the decomposition and its strengthenings

Benders cuts over a combinatorial subproblem — the lineage this solver belongs to, and the cut form it uses.

Hooker, J. N. & Ottosson, G. (2003). Logic-based Benders decomposition. *Mathematical Programming* 96, 33–60.
Codato, G. & Fischetti, M. (2006). Combinatorial Benders' cuts for mixed-integer linear programming. *Operations Research* 54(4), 756–766.

The survey all four strengthenings come from, including the initial cut pool seeded from the heuristic order.

Rahmaniani, R., Crainic, T. G., Gendreau, M. & Rei, W. (2017). The Benders decomposition algorithm: a literature review. *EJOR* 259, 801–817.

Choosing the deepest cut when the dual has many optima — and the free core point that makes it practical, which is the form implemented here.

Magnanti, T. L. & Wong, R. T. (1981). Accelerating Benders decomposition. *Operations Research* 29(3), 464–484.
Papadakos, N. (2008). Practical enhancements to the Magnanti–Wong method. *Operations Research Letters* 36, 444–449.

The conflict-graph inequalities the combinatorial row is the simplest case of.

Atamtürk, A., Nemhauser, G. L. & Savelsbergh, M. W. P. (2000). Conflict graphs in solving integer programming problems. *EJOR* 121, 40–55.

The hybrid genetic search used as the root incumbent — the only strengthening that gained instances.

Mecler, J., Subramanian, A. & Vidal, T. (2021). A simple and effective hybrid genetic search for the job sequencing and tool switching problem. *C&OR* 127, 105132.

The pricing problem for the position–configuration model is a set-union knapsack problem, the same class as the classical grouping pricer.

Goldschmidt, O., Nehme, D. & Yu, G. (1994). Note on the set-union knapsack problem. *Naval Research Logistics* 41, 833–842. Crama, Y. & Oerlemans, A. G. (1994). A column generation approach to job grouping for flexible manufacturing systems. *EJOR* 78, 58–80.

Dual stabilisation, the acceleration that damps the oscillation degenerate masters produce.

Lübbecke, M. E. & Desrosiers, J. (2005). Selected topics in column generation. *Operations Research* 53(6), 1007–1023.

The cut-selection framework the learned prototype follows.

Tang, Y., Agrawal, S. & Faenza, Y. (2020). Reinforcement learning for integer programming: learning to cut. *ICML*.

Where the position–transition construction generalises — any generalised travelling-salesman problem with overlapping clusters.

Pop, P. C., Cosma, O., Sabo, C. & Pop Sitar, C. (2024). A comprehensive survey on the generalized traveling salesman problem. *EJOR* 314, 819–835.

Thank you

Questions welcome

Shankar Narayanan

Advisors: Prof. Rafael Colares Prof. Annegret Wagler
Academic supervisor: Dr. Renaud Chicoisne

LIMOS — ISIMA, Clermont Auvergne INP

Backup slides

The other exact methods explored, and the validation

The obstacle

Every compact model surveyed relaxes to the counting bound $q = |U| - b$ or below. A branch-and-cut built on any of them is fast where q is exact and adrift where it is not.

Reformulating over **magazine configurations** attacks this from the other side. A configuration is a set $C \subseteq U$ with $|C| = b$; these become the columns, generated on demand. Three formulations were built.

	Column	Linear relaxation
PCF	configuration at a position	$= q$ with the counting rows, 0 without
PCF'	prefix form of PCF	$= q$, symmetry removed
PTF	<i>transition</i> between two	$\geq q$, and can exceed it

$y_C^k = 1$ if configuration C occupies position k ; $w_t^k \geq 0$ counts an insertion of t at position k .

$$\min \sum_{t,k} w_t^k$$

$$(P) \quad \sum_C y_C^k = 1, \quad k = 1, \dots, n$$

$$(Cov) \quad \sum_k \sum_{C \supseteq T_j} y_C^k \geq 1, \quad j \in J$$

$$(W) \quad w_t^k \geq \sum_{C \ni t} (y_C^k - y_C^{k-1}), \quad t \in T, k \geq 2$$

$$(T) \quad \sum_{k \geq 2} w_t^k + \sum_{C \ni t} y_C^1 \geq 1, \quad t \in U$$

The counting rows are load-bearing

Without (T) the relaxation is **zero** — one convex blend of configurations, repeated at every position, satisfies (P), (Cov) and (W) at no cost. With (T) it is exactly q . *Proved, and confirmed on all 2,108 recorded roots.*

$z_{C,C'}^k \geq 0$ is a column for the step from position k to $k+1$ leaving C and entering C' , at cost $d(C, C') = |C' \setminus C|$.

$$\min \sum_k \sum_{(C,C')} d(C, C') z_{C,C'}^k$$

$$(S) \quad \sum_{(C,C')} z_{C,C'}^k = 1, \quad k = 1, \dots, n-1$$

$$(F) \quad \sum_{(C,C'): t \in C'} z_{C,C'}^k = \sum_{(D,D'): t \in D} z_{D,D'}^{k+1}, \quad t \in T$$

(Cov) every job served at the head or tail of some step

$$(T) \quad \sum_k \sum_{(C,C'): t \notin C, t \in C'} z_{C,C'}^k + \sum_{(C,C'): t \in C} z_{C,C'}^1 \geq 1, \quad t \in U$$

Why this one can beat the counting bound

A column *carries a step*, so the relaxation cannot mix configurations without paying the transition. Hence $\text{LP}(\text{PTF}) \geq q$, and it can be strictly greater — the only construction in this work that provably exceeds q .

The pricing problem

Choosing the entering configuration is

NP-hard: pick b tools to collect per-tool duals plus a bonus for each job fully covered. PTF prices *coupled pairs*, so a node costs far more again.

Rows stay small

$O(n|T|)$ and fixed; only the columns are generated.

What was measured

- PCF' root = q on **all** 2,108 recorded runs — the proposition confirmed on the benchmark.
- PTF exceeds q on **2 of 232** roots, rising from 5 to 6 where the optima are 12 and 13.
- Nothing above nine jobs was closed: a Python pricer against compiled solvers.

Sound, and as it stands too rare and too small to close the certificate gap.

The question, raised by Prof. Colares. At a node the master has several admissible cuts: one per candidate order evaluated, the coverage row, and one per violated fractional point. A fixed rule decides. Would a learned policy do better?

A prototype was built following the cut-selection framework of **(Tang et al., 2020)**: selection as a Markov decision process — state is the relaxation point plus a candidate pool summarised by violation, support size, right-hand side and mean coefficient; reward is the rise in the dual bound; the policy is a linear softmax trained by REINFORCE.

Trained and evaluated on **knapsack cover cuts**, not on the SSP: the Benders solver needs a commercial solver the development environment did not have. 2,000 episodes at each of six sizes, one seed each, held-out evaluation.

n	learned	random	gain
10	1.3855	0.8734	+58.6%
20	0.7482	0.5021	+49.0%
26	0.5661	0.3279	+72.7%

Two things follow and only two: **the machinery learns**, and that is a statement **about knapsack cover cuts**. An appendix, not a method.

The reason is the same one the whole talk turns on

Cut selection chooses *among cuts a method can already generate*. Its established role in the literature is to pick well, not to make a family stronger.

In this campaign the available selections did not improve the terminal bound on a single instance — 67.5 million fractional cuts raised it on none. A selector cannot select a strong cut that does not exist.

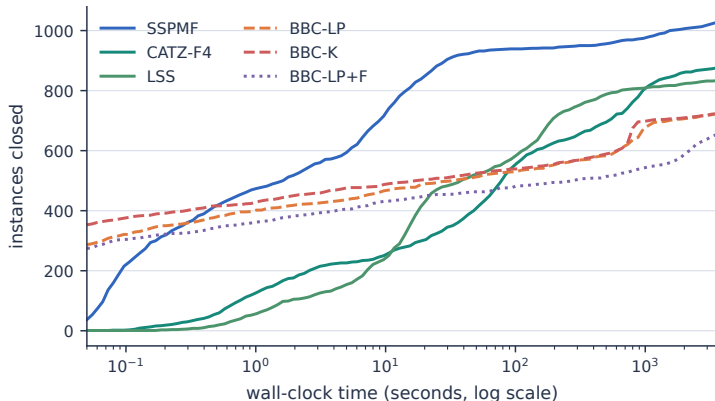
Where learning would be worth trying instead. Column selection for the position-indexed pricers, whose coupled-pair oracle is the measured bottleneck. The non-learned version of the same idea is automatic dual stabilisation.

Configuration	Catanzaro	Crama	Laporte3	Laporte4	Laporte5	Laporte7	All
SSPMF	62/160	68/160	340/340	330/330	162/340	66/80	1028/1410
CATZ-F4	44/160	44/160	340/340	330/330	74/340	44/80	876/1410
LSS	41/160	42/160	340/340	330/330	39/340	41/80	833/1410
BBC-LP	34/160	35/160	340/340	192/330	84/340	39/80	724/1410
BBC-LP+T	34/160	34/160	340/340	192/330	84/340	39/80	723/1410
BBC-K	35/160	35/160	340/340	190/330	83/340	39/80	722/1410
BBC-LP+F+H	39/160	37/160	316/340	157/330	97/340	40/80	686/1410
BBC-K+F	33/160	31/160	316/340	156/330	86/340	35/80	657/1410
BBC-LP+F	33/160	32/160	313/340	156/330	85/340	36/80	655/1410
BBC-LP+F+C	33/160	32/160	311/340	157/330	85/340	36/80	654/1410
BBC-LP+ACC	39/160	37/160	286/340	154/330	97/340	39/80	652/1410
BBC-LP+F+P	31/160	32/160	282/340	155/330	83/340	35/80	618/1410

The families *separate* the methods rather than rank them uniformly. Laporte4 splits them sharply — all 330 for every compact model, between 154 and 192 for Benders. And every configuration carrying +F sits below all three that do not separate at fractional points.

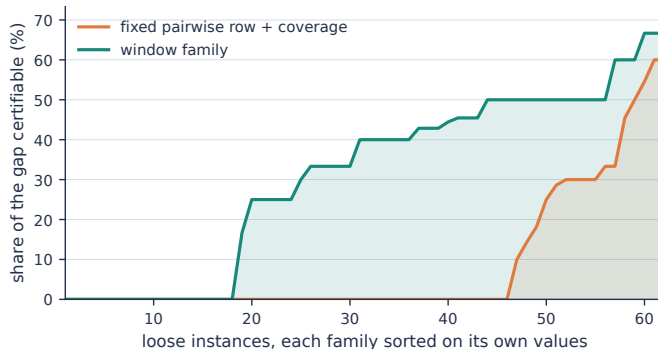
- **No two methods disagree.** 988 instances were closed by at least two methods, giving 41,514 method pairs that both proved an optimum. In none of them do the two values differ.
- **A fixed, audited denominator.** Every planned canonical pair counts once, certified or not; any non-optimal outcome counts as not closed. 16,920 planned outcomes, all present.
- **The identity of a run was tightened.** The historical runner keyed on file name and configuration alone, so repeated Crama names left 58 planned pairs unrecorded; they were completed under the same protocol before aggregation. The key is now (family, file, jobs, tools, capacity, configuration).
- **A predicted identity, confirmed on real data.** Empty-start and free-initial costs must differ by the initial load. SSPMF reports the latter; the difference equals b on 1,019 of the 1,028 instances it closed — and is smaller on exactly the nine whose optimum opens with a job needing $< b$ tools.
- **Every figure is regenerated from the raw ledger** by a checking script, and 58 independent mathematical checks re-run from the source data.

Backup — would a longer time limit change the story?



The counts would move, and they would not move equally for all methods — which is why the comparison is reported at one fixed limit and the conclusions rest on relaxation values rather than on solve counts.

Backup — the ceiling measurement, in full



Enumerating every sequence of the 62 loose cases in a fixed 81-instance sample at eight jobs gives the largest value each family could take on an integral schedule. The window family has roughly three and a half times the headroom of the pairwise rows.

That is a ceiling, not a bound — and the fractional row that implements it was then proved redundant. The gap between those two sentences is the honest result of this part of the work.